



Bienvenue sur Sakata

Ce portail est un sanctuaire numérique dédié à la préservation et à la transmission de l'identité culturelle du peuple **Sakata**.



Notre objectif est d'unir nos racines ancestrales avec la splendeur numérique moderne, en offrant à la fois des archives vivantes, des sagesses intemporelles et un forum d'échange intergénérationnel.

Suivant →

Le rôle `temp_admin` permet à un admin titulaire de déléguer ses pouvoirs pour 24 heures à un user de confiance, sans risquer une compromission permanente du contrôle. Le `temp_admin` agit comme un admin **sauf** sur deux opérations sensibles : il ne peut pas supprimer un autre admin, ni modifier son rôle.

SCHÉMA

Modèle de données

Enum + colonnes profiles

CODE

SQL

```
-- Ajout dans l'enum
ALTER TYPE user_role ADD VALUE 'temp_admin';

-- Trois colonnes pour le cycle de vie
ALTER TABLE profiles
  ADD COLUMN temp_admin_expires_at TIMESTAMPTZ,
  ADD COLUMN temp_admin_granted_by UUID REFERENCES profiles(id),
  ADD COLUMN temp_admin_original_role user_role;
```

Table d'audit temp_admin_grants

CODE

SQL

```
CREATE TABLE temp_admin_grants (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  recipient_id UUID REFERENCES profiles(id),
  granted_by UUID REFERENCES profiles(id),
  granted_at TIMESTAMPTZ DEFAULT NOW(),
  expires_at TIMESTAMPTZ NOT NULL,
  revoked_at TIMESTAMPTZ,
  revoked_by UUID REFERENCES profiles(id),
  reason TEXT,
  original_role user_role NOT NULL
);
```

API DB

Helpers SQL

FONCTION	RETOURNE TRUE SI
<code>is_admin_effectively(uid)</code>	role='admin' OU (role='temp_admin' ET expires_at > NOW())
<code>is_real_admin(uid)</code>	role='admin' uniquement
<code>is_admin_or_manager(uid)</code>	Précédent OU role='manager'
<code>can_modify_profile(actor, target)</code>	actor real_admin OU (actor admin_eff ET target NOT real_admin)

FLUX

Cycle de vie

- Admin titulaire ouvre `/admin/users`, clique sur l'icône éclair  du user cible.
- Modal saisit la raison (optionnelle), confirme.
- POST `/api/admin/temp-grants` :
 - Vérifie actor = real_admin
 - Vérifie target ≠ admin titulaire
 - Si target déjà temp_admin actif : revoke le grant courant
 - Insert audit row dans `temp_admin_grants`
 - UPDATE profile : role='temp_admin', expires_at=NOW()+24h, granted_by, original_role
- Pendant 24h, le user voit Command Center avec badge « Admin Temp · Xh restant ».
- À l'expiration ou révocation : profile remis à `original_role`, grant.revoked_at renseigné.

GARDE-FOUS

Restrictions du temp_admin

CE QU'UN TEMP_ADMIN NE PEUT PAS FAIRE

- Supprimer un autre administrateur titulaire
- Modifier le rôle d'un administrateur titulaire (rétrograder ou supprimer)
- S'octroyer à lui-même un autre temp_admin (anti-cascade)
- Prolonger sa propre durée

CE QU'IL PEUT FAIRE

- Modérer le forum et le chat
- Valider et publier les contributions
- Éditer tous les contenus
- Supprimer/modifier le rôle des managers, contributors, users
- Bannir des utilisateurs (sauf admins)

INTERFACE

UI

- **Admin titulaire** sur `/admin/users` : icône  pour promouvoir, icône  ambre pour révoquer.
- **Modal de promotion** avec récap et champ raison.
- **Badge dans la Navbar** pour le temp_admin actif : « Admin Temp · 21h 42min », pulse rouge à moins d'1h.
- **Colonne rôle** dans le tableau membres affiche le countdown sous le label.

HELPERS

Côté code TypeScript

CODE

TYPESCRIPT

```
import {  
  isTempAdminActive,  
  getEffectiveRole,  
  canModifyUser,  
  formatTempAdminRemaining,  
} from "@lib/constants/business";  
  
// Le profile retourné par AuthProvider contient désormais :  
// - role : rôle brut ('temp_admin' possible)  
// - effectiveRole : rôle calculé ('admin' si temp_admin actif)  
// - tempAdminExpiresAt  
// - isTempAdminActive  
  
const { role, effectiveRole, isTempAdminActive } = useAuth();  
  
// Pour les checks runtime : préférer effectiveRole  
if (effectiveRole === 'admin') {  
  // marche pour vrai admin ET temp_admin actif  
}
```

RÈGLES

Sécurité — RLS et garde-fous

RÈGLES NON NÉGOCIABLES

- Seul un VRAI admin peut accorder ou révoquer un temp_admin (RLS sur `temp_admin_grants`).
- Un temp_admin ne peut PAS toucher à un `profile.role = 'admin'` (vérifié côté UI + RLS via `can_modify_profile`).
- L'expiration est checkée à chaque requête via `NOW() > expires_at` — pas de cache, pas de latence.
- Toute action est traçable via `temp_admin_grants` (qui a été promu, par qui, quand, raison).

QUAND L'UTILISER

Cas d'usage

- **Voyage du fondateur** : déléguer à un manager pendant un déplacement de plusieurs jours.
- **Test de promotion** : essayer un manager comme admin avant promotion définitive.
- **Modération de crise** : apporter du renfort temporaire en cas de pic de signalements.
- **Couverture événement** : Audio Room en live qui nécessite plus de modérateurs.

À SAVOIR

Limitations connues

- Pas de notification push automatique au user promu (à venir avec PWA).
- Pas de notification 1h avant expiration (à ajouter).
- Pas de cron qui flush les grants expirés en DB — mais ils n'ont aucun effet runtime grâce aux helpers SQL.
- Un seul temp_admin actif par user (le nouveau revoke l'ancien).

SAKATA

© 2026 — Brume de la Rivière. Tous droits réservés.

CULTURE ECOLE LANGUE HISTOIRE FORUM